Demo: Attribute-Stream-Based Access Control (ASBAC) with the Streaming Attribute Policy Language (SAPL)

Dominic Heutelbeck
dheutelbeck@ftk.de
FTK - Forschungsinstitut für Telekommunikation und Kooperation e. V.
Dortmund, NRW, Germany

ABSTRACT

Traditional Attribute-Based Access Control (ABAC) implementations are based on a request-response protocol resulting in one decision for one authorization request. In stateful, session-based applications this may lead to polling the policy decision point (PDP) and policy information points (PIPs) referenced by policies. This leads to the well-known trade-offs between polling frequency and latency of updates becoming known to the client. Attribute-Stream-based Access Control (ASBAC) is an authorization model employing a publish-subscribe pattern protocol where a single subscription results in a stream of decisions dynamically updating based on the streams of changing attribute values, streams of changing policies, and PDP configuration. This demonstration will show the basic architecture of ASBAC based systems, show how the model is implemented by the Streaming Attribute Policy Language (SAPL) and its engine. The SAPL engine is available as Open Source for security researchers and practitioners. First, the overall architecture of the engine will be introduced, followed by a short demo of the publish-subscribe protocol. Afterward, several key features of SAPL are demonstrated in an authoring and administration environment. Next, a full-stack web application employing SAPL is demonstrated, showing how Policy Enforcement Points (PEPs) are established. The demonstration concludes with a demo of the SAPL tools for testing policies and calculating test metrics on policies.

CCS CONCEPTS

• **Security and privacy** → **Access control**; **Authorization**.

KEYWORDS

access control, data structures, policy languages, attribute-based access control, data streams

ACM Reference Format:
Dominic Heutelbeck. 2021. Demo: Attribute-Stream-Based Access Control (ASBAC) with the Streaming Attribute Policy Language (SAPL). In *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies (SACMAT '21), June 16–18, 2021, Virtual Event, Spain.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3450569.3464397

SACMAT '21, June 16–18, 2021, Virtual Event, Spain

ACM ISBN 978-1-4503-8365-3/21/06.
https://doi.org/10.1145/3450569.3464397

1 ATTRIBUTE STREAM-BASED ACCESS CONTROL (ASBAC)

This demonstration covers core features of the Streaming Attribute Policy Language (SAPL) [2–4] and engine which is implementing Attribute Stream-Based Access Control (ASBAC) [1]. The engine and server implementations are available as Open Source projects for researchers and practitioners. Thus, the audience can directly access the engine and code online. As the event is held virtually the interaction is limited, however the audience is invited to ask for changes to policies and tests at different stages of the demo.

The novelty of the demonstrated system lie in:

- Publish-subscribe ABAC system
- Easy to use policy engine with data-stream semantics
- Parameterized Policy Information Points and attributes
- Unit test like approach for testing policies

ASBAC [1] introduces a functional architecture, as illustrated in Figure 1. It shares some key aspects with the architectures of both XACML and NGAC, with a policy decision point (PDP) in the center, responsible for interpreting policies and making authorization decisions. The primary difference to the previously mentioned architectures is not the structure of components, but the way they communicate. In XACML and NGAC a single authorization request made by the PEP to the PDP results in several requests and responses to and from the connected components for accessing policies, configuration, and external attributes.

In ASBAC the authorization request is replaced by an authorization subscription to the PDP (2). The PDP in turn subscribes to the PDP configuration (3). The configuration contains fundamental parameters for the decision making (e.g., the default policy combining algorithm selected for the deployment) or parameters for PIPs connected to the PDP (e.g., access tokens for message brokers or RESTful APIs). Also, the PDP makes a subscription to the policy retrieval point, parameterized by the authorization subscription. The PRP will return all policies (policy sets, rules if applicable) to be evaluated for the authorization subscription, and sends a new set of matching policies if they change due to policy management activities of the policy administration point. Each time the PDP has a new set of configuration and matching policies, it starts evaluating the policies. During the evaluation, when the PDP encounters references to policy information points, the PDP subscribes to them (5) as encoded in the policies and aggregates incoming attributes from the PIPs to calculate the most recent decision (6), which is forwarded to the PEP if it differs from the most recent previous decision (7). The PEP then enforces the decision (8) and, if applicable, performs the requested action. Then it provides the client application with access to the return value, if present, of the action (9)

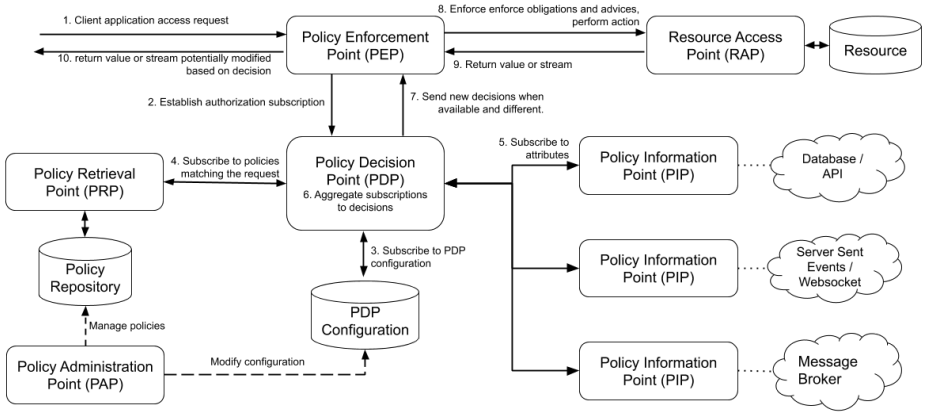

Figure 1: ASBAC Functional Architecture

which may be altered before being delivered to the client application (10), if this is indicated by the decision. The architecture should at every step be implemented using non-blocking asynchronous code. The functional model as presented in [1] is not specifying a policy language but covers the fundamental algorithms for stream-based PDPs and PEPs.

2 DEMO SCENARIO 1: THE SAPL PROTOCOL

The Streaming Attribute Policy Language (SAPL), previously called Structure and Agency Policy Language [**?**]. Uses a protocol similar to XACML for communication between a PEP and a PDP. The schemata of the requests/subscriptions and decisions have similar contents. However, XACML is XML-based and uses a request-response scheme and SAPL is using JSON and a publish-subscribe scheme.

```
{
  "subject" : {
                    "username" : "alice",
                    "id"       : 1234321
              },
  "action"  : "SUBSCRIBE",
  "resource": {
                    "type" : "mqtt",
                    "topic": "track/987/location"
              }
}
```

Listing 1: Example SAPL subscription

```
{
  "decision"   : "PERMIT",
  "obligations": [{ "action": "logAccess" }],
  "advices"    : [{
                    "action": "sendMail",
                    "to"    : "foo@bar.baz"
                  }]
}
```

Listing 2: Example SAPL decision event

Typically a PDP server will accept HTTP POST requests with a subscription and reply with decisions events sent as server sent events.

As a demonstration, a SAPL PDP server will be used and by using `curl` a subscription is sent and the event stream of decisions can be observed.

3 DEMO SCENARIO 2: POLICY LANGUAGE AND AUTHORING TOOL

The second demo scenario will use the SAPL Server CE application and its authoring tools to illustrate core features of the language.

A SAPL document with a policy policy matching the example subscription above may be expressed as follows:

```
policy "example1"
permit resource.type == "mqtt" & action == "SUBSCRIBE"
where
  subject.id.<profile.clearanceLevel> >= 5;
  subject.id.<physicalAccessControl.checkedIn>;
obligation
  { "action": "logAccess" }
advice
  {
        "action": "sendMail",
        "to"    : "foo@bar.baz"
  }
```

Listing 3: Example SAPL policy

In this example policy, `subject.id` evaluates to the number 1234321. By appending `.⟨profile.clearanceLevel⟩`, the policy engine is instructed to pass the preceding result (1234321) to the PIP, or *attribute finder*, with the name "profile" to retrieve the *attribute* "clearanceLevel". The PIP in turn subscribes to the attribute. In the next line, an attribute finder subscribes to data published by the physical access control system which registers, if users have physically checked into the workplace. Thus, the policy permits access, if the subject has a sufficient security clearance and is on the company premises under physical access control. Attribute finders have to be preregistered to the PDP. If no matching PIP is present, an evaluation error occurs. In case a matching policy evaluates to *PERMIT* or *DENY*, the optional `obligation` and `advice` blocks are evaluated, which contain expressions that evaluate to arbitrary JSON values, which are then added to the authorization sent to the PEP. These events are sent to the subscribing PEP, whenever a new different authorization event occurs based on the latest PIP data.

```
set "example2"

deny-unless-permit

for resource.type == "aType"

var elevated_function = "manager";

policy "example2.1"
permit subject.function == elevated_fucntion

policy "example2.2"
permit action == "read"
transform resource |- filter.blacken
```

Listing 4: Example SAPL policy set

Optionally a policy set may declare a policy set target expression after the keyword `for` which is used in the same way as the target statement of documents containing only a policy. The policy set is only evaluated if the policy set target-statement evaluates to *true* for a given subscription. Finally, before defining the individual policies arbitrary variables may be declared to be used during the evaluation of the following policies.

```
policy "administrator access to patient data"
permit action.java.name == "findById"
where
  "ADMIN" in subject..authority;
obligation
  {
    "type"    : "logAccess",
    "message" : subject.name + " has accessed patient
         data (id="+resource.id+") as an administrator."
  }
transform
  // templating and filtering with text blackening
  resource |- {
               @.icd11Code : blacken(2,0,"\u2588"),
               @.diagnosisText : blacken(0,0,"\u2588")
             }
```

Listing 5: SAPL policy with transformation, templating and filtering

Based on the patient data example the demo shows how to write policies that allow for resource transformation.

```
policy "permit on presence and with valid certificate"
permit resource.type=="realtime"
where
  subject.id.<presence.onPremise>;
  subject.ethereumAddress.<certification.
       holdsValidCertificate(resource.requiredCertificate
       )>;
```

Listing 6: SAPL policy with parameterized attribute access

Finally, the demo shows how parameterized attributes can be used in SAPL. The policy above fetches a flag indicating if the subject holds a certificate of a given type. Another use is including semantic attributes. e.g., a social network with group types where different types imply differing access rights. An attribute to fetch group membership with a given type is hard to express in XACML, but easy to implement in SAPL, especially if the different types are not known a priori.

4 DEMO SCENARIO 3: FULL-STACK APPLICATIONS AND DECLARATIVE POLICY ENFORCEMENT POINTS

The third demo shows full-stack web applications built with Java, Thymeleaf, and Vaadin.

The demo will show how Policy Enforcement Points can be set up automatically and manually in applications, how to establish handlers for obligations and advice, and how to implement simple Policy Information Points.

Besides, it will demonstrate the resource transformation feature of the SAPL policy engine where policies can manipulate the data of a resource. In this case, it is used to blacken out medical information

Finally, the streaming aspect of the policy enforcement will be demonstrated by using time-based policies updating decisions without polling the PDP.

The demonstration will feature a demonstration of the applications, then presenting key code segments and the policies enforced.

5 DEMO SCENARIO 4: TESTING OF POLICIES AND POLICY COVERAGE

The fourth and final scenario of the demos will demonstrate the testing fixtures of the SAPL Engine, enabling unit test style tests of policies and test coverage metrics and visualization.

Policy tests are written using Java. The SAPL test libraries can be used to mock remote Policy Information Points and time-based behavior.

The demonstration will show how tests for individual policies and a set of policies to be published are written and how the tooling generates coverage metrics and annotated policy documents

REFERENCES

[1] Dominic Heutelbeck. 2019. Attribute Stream-Based Access Control (ASBAC) - Functional Architecture and Patterns. In *Proceedings of the 2019 International Conference of Security and Management (SAM'19)* (2019-07-29).

[2] Dominic Heutelbeck. 2019. SAPL - Structure and Agency Policy Language. https://github.com/heutelbeck/sapl-policy-engine/blob/master/sapl-documentation/src/asciidoc/sapl-reference.adoc accessed 2019-05-10.

[3] Dominic Heutelbeck. 2019. SAPL Policy Engine. https://github.com/heutelbeck/sapl-policy-engine accessed 2019-05-10.

[4] Dominic Heutelbeck. 2019. The Structure and Agency Policy Language (SAPL) for Attribute Stream-Based Access Control (ASBAC). In *Proceedings of the ETAA 2019 : 2nd International Workshop on Emerging Technologies for Authorization and Authentication.*